

Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Sensor Network Technologies and Sensor Fusion

3D Video Object Detection using NNs
Stochastics Recap

Oct 23, 2025

Welcome to the lecture!

Prof. Dr.-Ing. Björn Giesler

Technische Hochschule Ingolstadt / Fraunhofer IVI

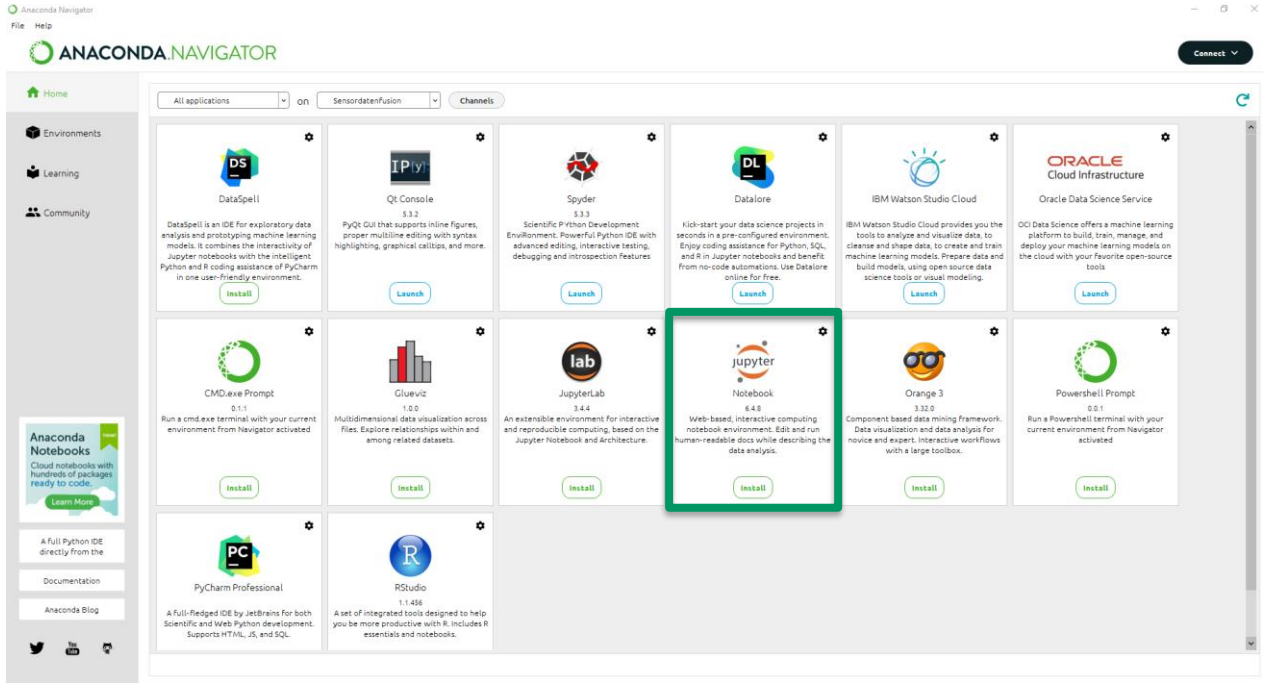
Autonomous, Cooperating Systems

Email: bjoern.giesler@thi.de

- **Please register for the lecture on Moodle!**
  - Link: https://moodle.thi.de/course/view.php?id=10580
  - Key: **SensNT+DF#WS25**
- **News on Moodle:**
  - Slides for the lecture (upcoming lectures marked as WIP and are subject to change!)
  - Formulary, Exercises (Solutions provided after discussion in the lecture)
  - Exercises are optional, one task / sheet per week, but are highly recommended.
    They can be handed in for correction.
  - I will *try* to publish exercises before Wednesday‘s lecture so that initial questions can be discussed on Thursday.
- **Exam: Written exam, 90 minutes**
- **Consultation hours: Please arrange via e-mail to bjoern.giesler@thi.de.**
- **Please also don‘t hesitate to send questions, lecture feedback, change requests, etc.!**

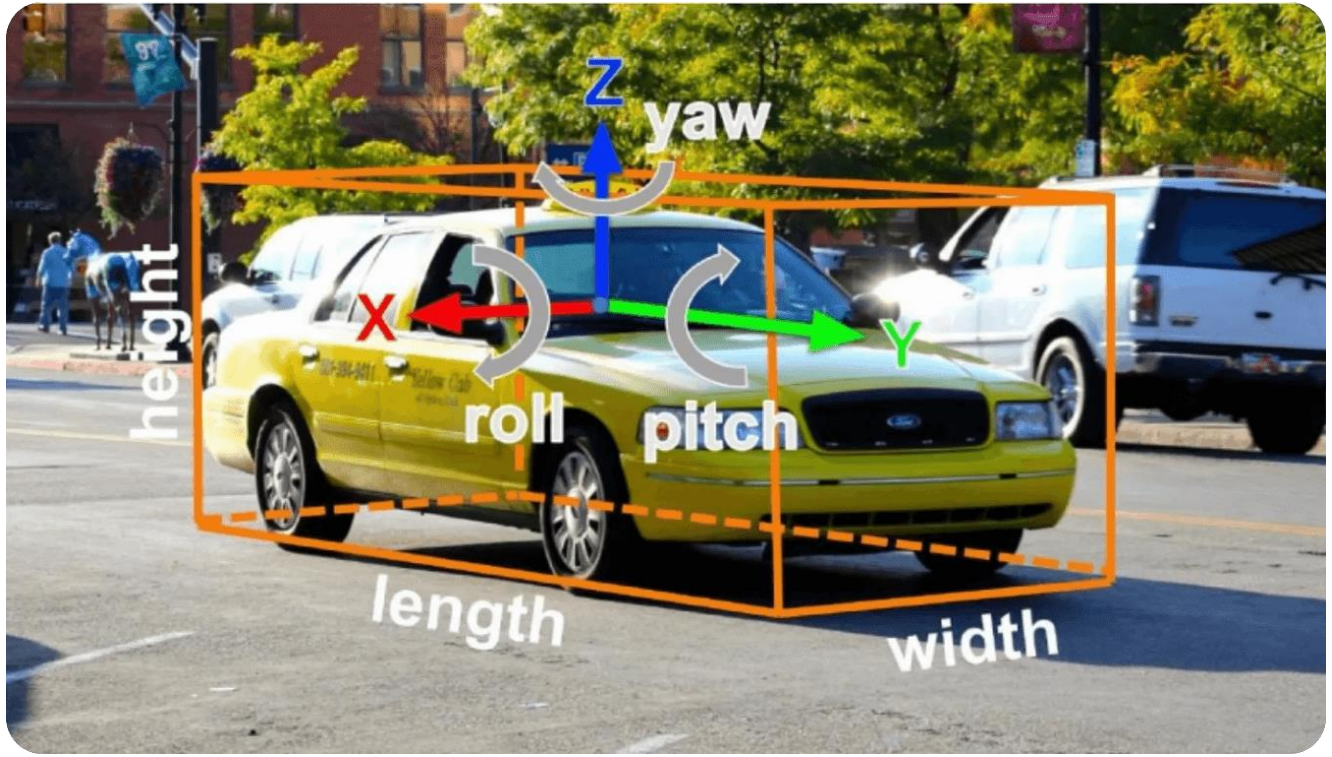
Software: Python/Anaconda

- For the ML examples we will use today, will use the Jupyter Notebook environment.
- It is recommended you install the Anaconda Python environment and distribution, that installs Jupyter as one of its environments.
- You can find it at https://www.anaconda.com/.



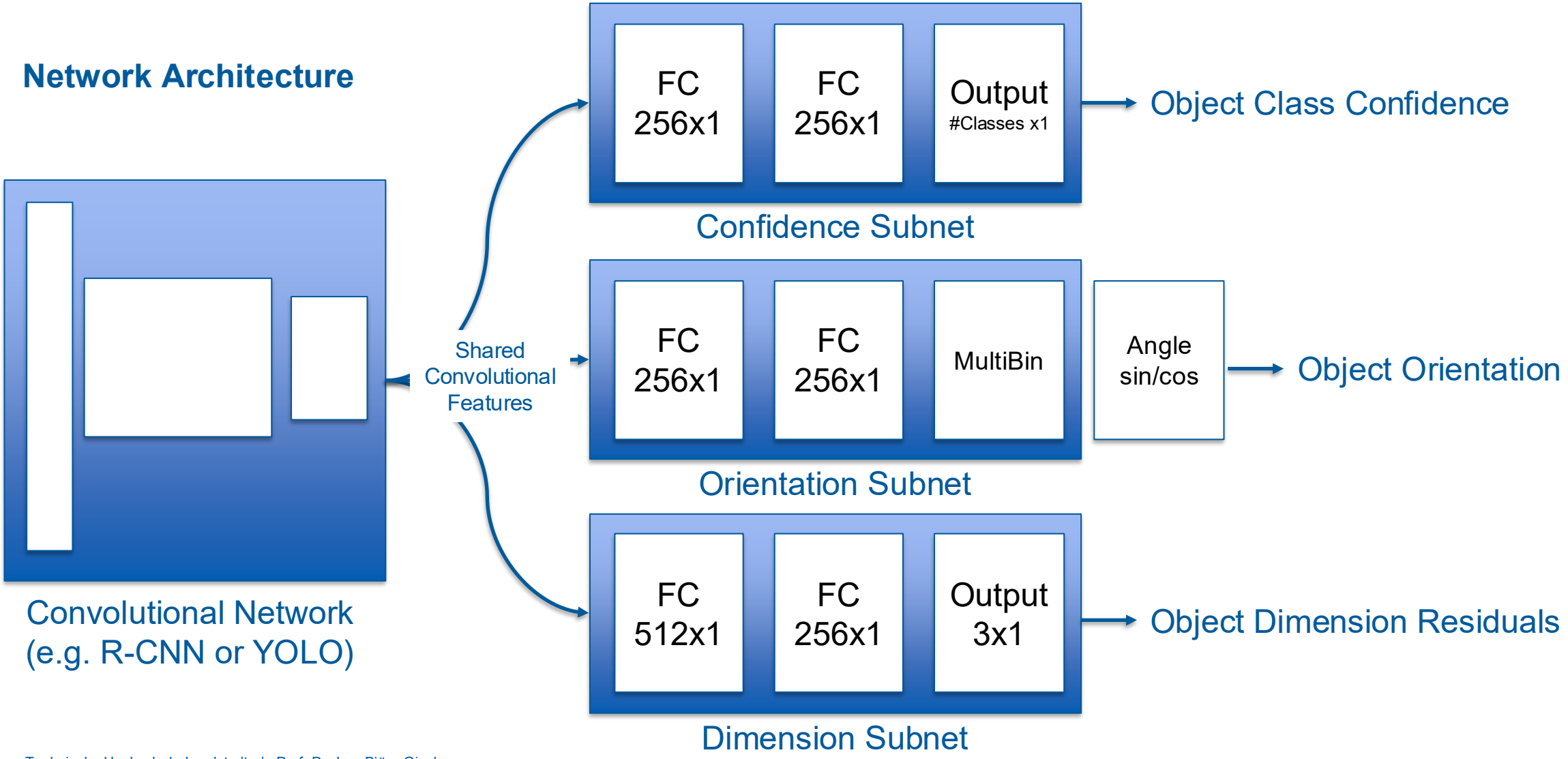
Machine Learning: 3D from 2D

While YOLO gave us $x, y, w, h, \text{confidence}$, 3D objects need more parameters:



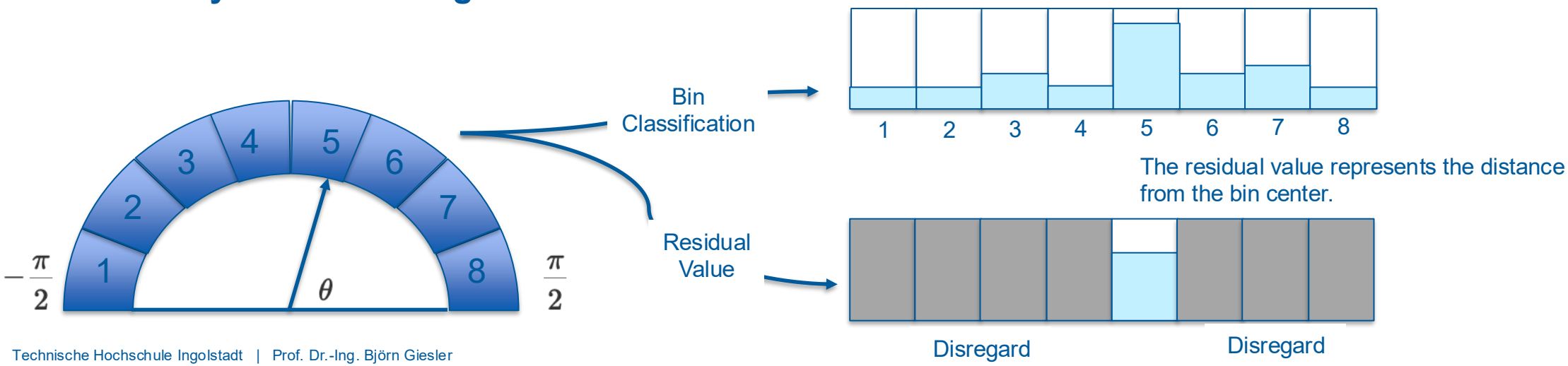
The 3D bounding box is defined by $\underbrace{\mathrm{T} = (t_x, t_y, t_z)^\top}_{\text{Center}}, \underbrace{\mathrm{D} = (d_x, d_y, d_z)^\top}_{\text{Dimension}}, \underbrace{R(\theta, \phi, \alpha)}_{\text{Rotation}}$.

Network Architecture



Multi-Bin Orientation Estimation

- **A Multi-Bin NN is a type of regression architecture commonly used for orientation estimation.**
- **It divides a continuous variable — typically an angle or pose — into several discrete bins, and then predicts both:**
  - Which bin the true value belongs to (classification), and
  - How far inside that bin the value lies (regression).
- **This approach helps the network handle periodic or continuous quantities (like angles) more effectively than direct regression.**



Multi-Bin Orientation Estimation

- **How to encode the residual angle $\delta_i = \theta_{\text{obj}} - \theta_{\text{bin}}$?**
- **The easiest way is to simply encode the angle directly.**
- **This leads to unstable predictions due to direct influence of network noise and uncertainty.**
- **A more robust solution is to**
  - encode $\sin(\delta_i), \cos(\delta_i)$ instead, receiving $\hat{s}_j, \hat{c}_j$ as the prediction for detected object O_j ,
  - enforce unity of the resulting 2-vector $$\sqrt{\sin(\delta_i)^2 + \cos(\delta_i)^2} = 1 \Longrightarrow \begin{bmatrix} \hat{s}'_j \\ \hat{c}'_j \end{bmatrix} = \frac{1}{\sqrt{\hat{s}_j^2 + \hat{c}_j^2}} \begin{bmatrix} \hat{s}_j \\ \hat{c}_j \end{bmatrix}$$
  - and finally obtaining the angle from $\hat{\theta}_{\text{obj},j} = \hat{\theta}_{\text{bin},j} + \sin^{-1} \hat{s}'_j$.

Multi-Bin Orientation Estimation

- **For N bins, the output for the network is of size 3xN:** $[p_1, \ldots, p_N, s_1, \ldots, s_N, c_1, \ldots, c_N]$ **where** p_i **are the bin probabilities and** s_j, c_j **are sines and cosines of the residuals.**
- **The error function is represented as a combination:** $E = E_{\text{bin}} + \lambda \cdot E_{\text{residual}}$ **, where**
  - E_{bin} is the binning error and
  - $E_{\text{residual}} = \cos(\theta_{truth} - \hat{\theta}_{bin} - \hat{\delta})$ is the error obtained from the angle calculated from predicted bin and residual.
  - A stabilizing extension of the algorithm is *overlapping the bins*, representing each angle with multiple (e.g. 2) bin / residual combinations, and interpolating between the two.

Method	Pros	Cons
Direct angle regression	Simpler Outputs continuous values directly	Poor handling of periodicity Struggles with multimodal distributions
Multibin network	Handles periodicity (0°=360°) More stable learning Better accuracy	Slightly more complex Bin choice matters

Learning Dimension

- **Dimension is tied strongly to object class. Cars, trucks, bicyclists are a certain size, even humans.**

- **We therefore only learn the residual $\mathbf{\Delta}_i$ to the mean dimensions $\bar{\mathbf{D}}_\mathrm{c} = \frac{1}{N_\mathrm{c}} \sum \begin{bmatrix} d_{x,\mathrm{c},i} \\ d_{y,\mathrm{c},i} \\ d_{z,\mathrm{c},i} \end{bmatrix}$ for object class c:**

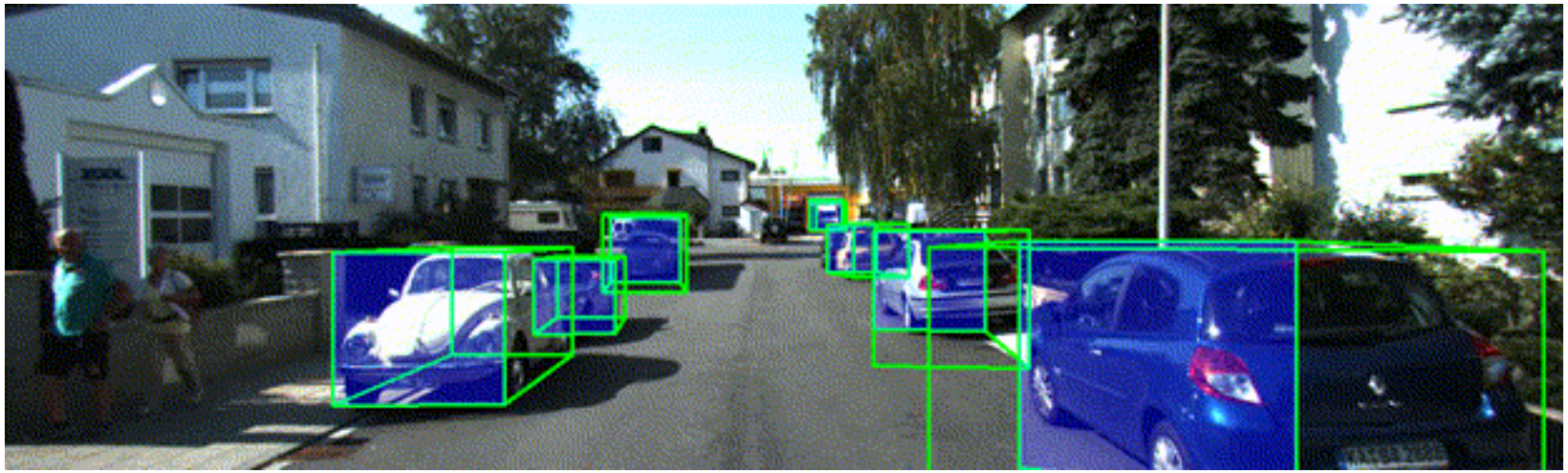
$$\mathbf{\Delta}_i = \mathbf{D}_i - \bar{\mathbf{D}}_\mathrm{c}$$

with the Error or Loss function $E_\mathrm{dims} = \frac{1}{n} \sum (\mathbf{D}_{\mathrm{truth},i} - \bar{\mathbf{D}}_\mathrm{c} - \hat{\mathbf{\Delta}}_i)$.

- **Using the obtained size and rotation, the full 3D information can be obtained with the following logic:**
  - There are 4096 total combinations of corner matches.
  - Assuming the object is always upright,
    - Top and bottom corners of the bounding box can only correspond to top and bottom corners of the object, reducing the number of options to 1024.
    - Left and right corners of the bounding box can only correspond to left and right corners of the object, reducing to 256.

Sensor Networks and Sensor Fusion

3D from 2D



Results (slowed down for visualization)
From sample implementation in [2]

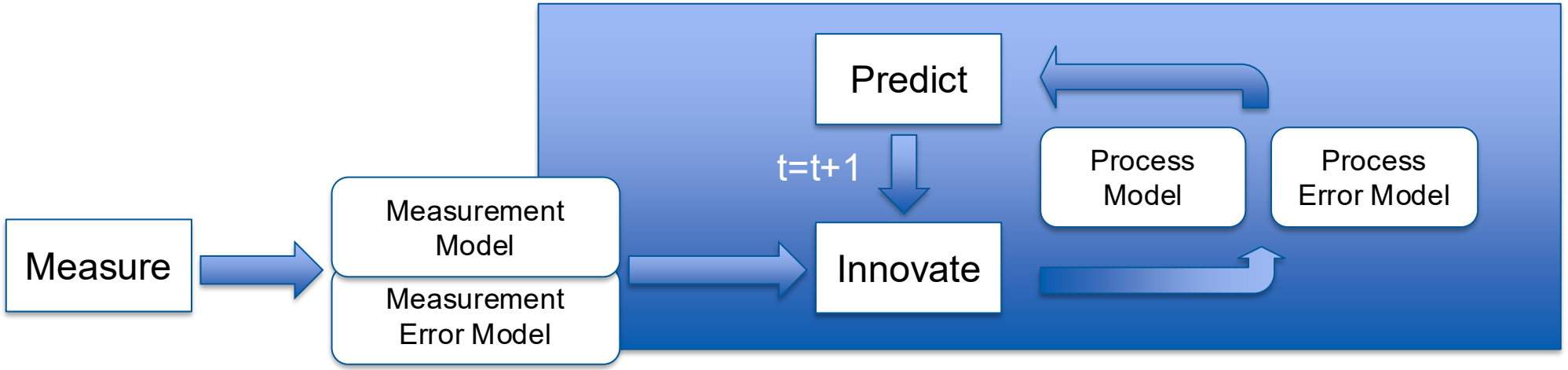


Features that contribute most to orientation estimation

System State Modeling

- **In the last lecture, we have looked at**
  - R-CNNs and YOLO as two of many ways of classifying and detecting objects from images,
  - A method to refine 2D bounding boxes into 3D objects, relating back from projective 2D space into the real world environment.
- **We have seen from the programming examples (IMU, face detection) and videos that our measurements are often quite noisy – they exhibit residual error that varies over time.**
- **The complementary filter simply ignored the noise and only got rid of bias.**
- **Linear camera calibration got rid of the residual error by minimizing its square all at once.**

- **Often we do not have access to the full measurement series, e.g. if we are monitoring a process over time.**



- **However we can use assumptions about how the process *should* behave, and about the nature of the error influences, to minimize the error nonetheless.**
- **In the next lectures, we will examine:**
  - Probability distributions that we can use to derive statistical properties about process and noise
  - Deriving optimal state estimates from individual (iterative or simultaneous) observations.

Remembering the Complementary Filter we introduced in Lecture 1:

- **We can think of the development of the state of a system that follows a linear, time-invariant model like this:**

$$\mathbf{x}_n = \mathbf{A}\mathbf{x}_{n-1} + \mathbf{B}\mathbf{u}_{n-1}$$

where

- x is the state vector,
- A is the system model, i.e. “the way things change in the system over time if there is no input“,
- u is a vector of inputs to the system,
- and B is the way how these inputs are influencing our system state.

- **If there were no noise and we could just integrate the gyro, our model can be written as**

$$\underbrace{\begin{pmatrix}\hat{\alpha}_n\\ \hat{\beta}_n\end{pmatrix}}_{\mathbf{x}_n} = \underbrace{\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix}\hat{\alpha}_{n-1}\\ \hat{\beta}_{n-1}\end{pmatrix}}_{\mathbf{x}_{n-1}} + \underbrace{\begin{pmatrix}\Delta t & 0\\ 0 & \Delta t\end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix}\dot{\alpha}_{n-1}\\ \dot{\beta}_{n-1}\end{pmatrix}}_{\mathbf{u}_{n-1}}$$

- **However, we know that that is not correct, so we introduce a correction term...**

$$\mathbf{x}_n = \mathbf{A}\mathbf{x}_{n-1} + \mathbf{B}\mathbf{u}_{n-1} + \mathbf{L}(\mathbf{x}_{n-1} - \mathbf{x}_{\mathrm{meas},\mathbf{n-1}})$$

...again with a model L that governs how the correction term applies to the system state.

- **We can write this now as**

$$\underbrace{\begin{pmatrix}\hat{\alpha}_n\\ \hat{\beta}_n\end{pmatrix}}_{\mathbf{x}_n} = \underbrace{\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}}_{\mathbf{A}} \underbrace{\begin{pmatrix}\hat{\alpha}_{n-1}\\ \hat{\beta}_{n-1}\end{pmatrix}}_{\mathbf{x}_{n-1}} + \underbrace{\begin{pmatrix}\Delta t(1-k) & 0\\ 0 & \Delta t(1-k)\end{pmatrix}}_{\mathbf{B}} \underbrace{\begin{pmatrix}\dot{\alpha}_{n-1}\\ \dot{\beta}_{n-1}\end{pmatrix}}_{\mathbf{u}_{n-1}} + \underbrace{\begin{pmatrix}k & 0\\ 0 & k\end{pmatrix}}_{\mathbf{L}} \begin{pmatrix}\hat{\alpha}_n - \hat{\alpha}_{\mathrm{acc},n-1}\\ \hat{\beta}_n - \hat{\beta}_{\mathrm{acc},n-1}\end{pmatrix}$$

- **The above form is known as a *Luenberger Observer* and is a classic in control theory.**
- ***Neither of the above terms will ever be accurate.***
- **Let‘s find out how to model that inaccuracy and reduce it as best we can.**

Stochastics Recap

Example: Random Experiment

- Throwing a fair dice: Six possible outcomes ω_i from sample space

$$\Omega = \{"1", "2", "3", "4", "5", "6"\}$$

Random Variable / Realization of a Random Variable:

- A **function** $\mathrm{X}(\omega)$, which assigns a real number x to every outcome $\omega \in \Omega$, is called **random variable**, so we have

$$x = X(\omega)$$

- The real number x for one specific outcome is denoted as **realization of a random variable** $\mathrm{X}(\omega)$.

Example 1:

- Throwing a dice: Possible outcomes ω_i

$$\Omega = \{"1", "2", "3", "4", "5", "6"\}$$

- Number of points on the dice (realization) $x_i = X(\omega_i)$

Example 2:

- Throwing a coin : Outcomes ω_i are heads and tails
- Assign a real number to every outcome, i. e. head $\rightarrow 0$, tail $\rightarrow 1$, i.e. $X(\omega) \in \{0, 1\}$

Events

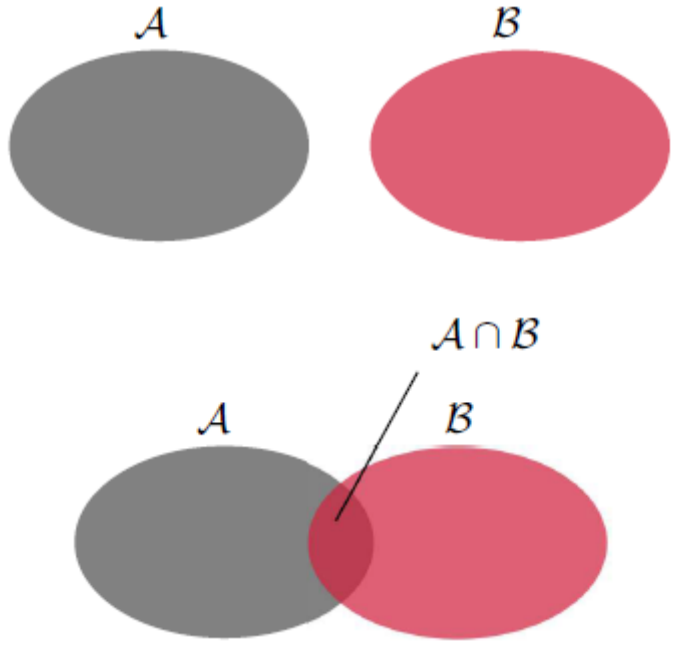
- Sample Set
  $$\Omega = \{\omega_1, \omega_2, \ldots, \omega_L\}$$
- Define an Event $A = \{a_1, a_2, \ldots, a_M\} \subseteq \Omega$ with $M \leq L$
- Certain event $A = \Omega$
- Impossible event $A = \emptyset$
- Complementary event A^c
- Disjoint events: $A \cap B = \emptyset$ (event A and B can not occur jointly)

Example: Throwing a dice

- Sample set of possible outcomes ω_i
  $$\Omega = \{"1", "2", "3", "4", "5", "6"\}$$
- Define events:
  - Throwing a „3“, $A = \{3\}$
  - Throwing an even number $A = \{2, 4, 6\}$
- Complementary event for $A = \{2, 4, 6\}$ is $A^c = \{1, 3, 5\}$
- Disjoint event $A \cap B$ with $A = \{2, 4, 6\}$ and $B = \{7\}$

Events

- Sample Set
  $$\Omega = \{\omega_1, \omega_2, \ldots, \omega_L\}$$
- Define an Event $A = \{a_1, a_2, \ldots, a_M\} \subseteq \Omega$ with $M \leq L$
- Certain event $A = \Omega$
- Impossible event $A = \emptyset$
- Complementary event A^c
- Disjoint events: $A \cap B = \emptyset$ (event A and B can not occur jointly)



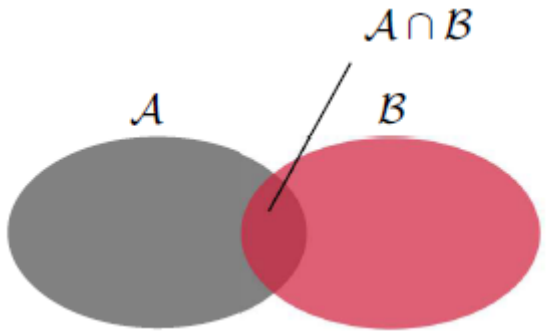
Disjont events (top) and non-disjoints events (bottom)

Frequency of Events and Probability

- Frequency $h(A)$ and probability $P(A)$

$$P(A) = \lim_{n\to\infty} h_n(A) \quad \text{with} \quad h_n(A) = \frac{\text{number of experiments with outcome A}}{n}$$

- Axioms of Kolmogorov
  - Probability for an event with an outcome between 0 and 1, i. e. it holds
    $$0 \leq P(A) \leq 1$$
  - For an event that is certain to occur, we have $P(\omega) = 1$.
  - The probability that any event from the sample set occurs is certain, i.e. $P(\Omega) = 1$.
  - For disjoint events $A \cap B = \emptyset$ we have that $P(A \cap B) = P(A) + P(B)$.



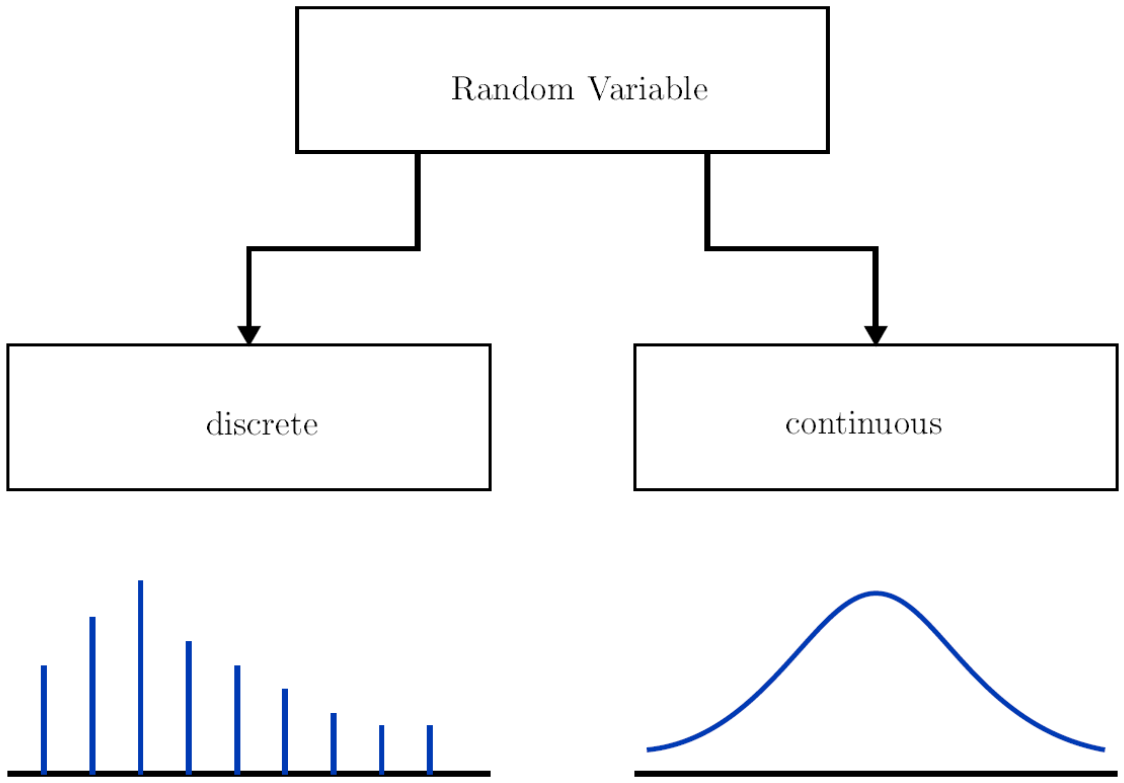
Union of non-disjoint events

Conclusions:

- Probability of a complementary events A^c is $P(A^c) = 1 - P(A)$.
- Probability of the impossible event is $P(\emptyset) = 0$.
- Union of non-disjoint events $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

Discrete vs. Continuous random Variables

- Discrete random variable: Only discrete (Throwing dice/a coin)
  - Discrete probability density function
- Continuous random variable: Realization can take arbitrary values (Analogous output voltage of a sensor)
  - Continuous probability density function



Probability density functions of discrete and continuous random variables



Technische Hochschule Ingolstadt

Fakultät Elektro- und Informationstechnik

Thank you very much!